

Block Round

项目的数学原理

圆、椭圆、球体和椭球体生成器的数学基础技术与教学文档，使用 *Minecraft* 方块纹理渲染。对每个概念给出形式定义、几何论证、与项目源代码的直接对应关系，以及在游戏内构建命令（`/fill`、`/clone`、`//sphere`）中的实用翻译。

涉及领域：解析几何 · 离散光栅化 · 数字拓扑 · 积分学 · 线性代数 · 三角学 · 物品栏算术 · 纹理映射 · 八面体对称性。

目录

1	概述：问题的数学本质	3
2	坐标系与离散体素网格	4
3	基本方程：圆与椭圆	4
4	欧氏算法（距离测试）	5
5	Bresenham 算法（整数中点）	6
5.1	圆形情况	6
5.2	椭圆情况（两个区域）	7
6	阈值算法（角点覆盖）	7
7	渲染模式：实心、细、粗	8
7.1	实心	8
7.2	细（轮廓）	8
7.3	粗	8
8	离散面积计算	8
9	从 2D 到 3D：椭球方程	9
10	体素化与体积计算	10
11	几何切割：平面与 45° 对角切割	11
12	粗 3D 模式：通过切片的三维化	11
13	3D 相机：球坐标	12
14	自动缩放：包围球与 FOV	13
15	阴影与纹理乘法	13
16	过渡：从连续数学到游戏内建造	13
17	Minecraft 物品栏算术	15
17.1	单格、单堆、满包	15
17.2	储存：箱子、双箱、潜影盒	15
17.3	参考表：标准球	16

18 高亮覆盖层与裸露面计数	16
18.1 什么算裸露面	16
18.2 边界检测公式	16
18.3 裸露面计数作为生存启发法	17
19 逐层建造 (水平分解)	17
20 通过八面体对称性的优化 (O_h)	18
20.1 具体命令序列	18
20.2 时间成本对比	19
21 方块纹理与视觉构图	19
21.1 六个面与 UV 映射	19
21.2 方块家族与色调	20
21.3 通过方块混合的渐变	20
22 结论	21
参考文献	21
附录 A: 建造参考表与示例	24

1. 概述：问题的数学本质

Block Round 是一个生成圆角形状的工具，使用 *Minecraft* 方块纹理渲染：2D 中的圆与椭圆，3D 中的球体与椭球体。核心问题属于离散光栅化类：给定一条在实数 $\mathbb{R}$ 上解析定义的曲线或曲面，确定规则网格（2D 中的像素、3D 中的体素）的哪些单元应被标记以表示它。Block Round 中每个被标记的单元额外获得来自 *Minecraft* 调色板的六面纹理（草方块、圆石、橡木板、石英、钻石等），因此结果图形不仅是数学轮廓，更是用户可在生存模式中复制、可在创造模式中通过 `/fill` 构造、或通过 *Sponge Schematic v2* (`.schem`) 导入 *WorldEdit*、*Litematica* 或 *MCEdit* 的可构建结构。

该问题没有唯一解。不同的包含准则产生不同的离散轮廓，每个准则有自己的数学论证。因此项目实现了三种不同的 2D 算法（欧氏、Bresenham、阈值）、三种渲染模式（实心、细、粗）和三种 3D 阴影样式。算法选择不仅取决于视觉平滑度，还取决于用户需要采集的方块数——直径 $D = 20$ 的球在欧氏算法下需要 4189 个方块，在阈值算法下高达 4322 个。

执行完全在浏览器中进行，无服务器组件，因此对数学表述施加了额外的计算效率要求（ $D = 32$ 的钻石球必须以 60 fps 渲染，含 17156 个有纹理体素）。所涉及的理论领域包括：

- 圆锥曲线与二次曲面的解析几何；
- 整数算术的增量算法（Bresenham）；
- 数字拓扑（4-、6- 与 26-连通邻域）；
- 积分学与计数测度；
- 线性代数（切割平面、透视投影）；
- 三角学与球坐标；
- 组合物品栏算术与群对称性。

相对于姊妹项目 Pixel Round 的新增内容。 Block Round 增加了五个仅当渲染单元变为 *Minecraft* 方块时才出现的数学考量：(i) 每个体素六个面的 UV 映射；(ii) 生存模式物品栏的算术（每堆 64 个、单箱 27 格、双箱 54 格）；(iii) 用于建造时统计裸露面的高亮覆盖层；(iv) 由于现实建造逐层进行，将 3D 形状分解为水平层；(v) 利用八面体对称群 O_h 通过 `/clone` 将人工放置方块成本降低 8 倍。

2. 坐标系与离散体素网格

画布是 2D 中 $G_x \times G_y$ 单元的网格，3D 中扩展为 $G_x \times G_y \times G_z$ 体素。每个单元 (i, j) 占据单位正方形 $[i, i+1] \times [j, j+1]$ ；3D 中每个体素占据单位立方体。在连续代码中，单元的中心位于 $(i + \frac{1}{2}, j + \frac{1}{2})$ 。这个约定至关重要：将圆周与角点 (i, j) 或中心进行比较会改变哪些单元属于形状，进而决定用户必须放置哪些方块。

$$\text{单元 } (i, j) \text{ 的中心} = (i + \frac{1}{2}, j + \frac{1}{2})$$

对于直径 D 的形状，代码将网格扩展为每轴 $D + 2$ 个单元（每侧一格余量）。这确保极端像素（北、南、东、西）永远不会落在矩阵边界上。形状的中心位于 $(G_x/2, G_y/2)$ ，半轴为 $r_x = D/2$ 和 $r_y = D/2$ 。3D 中相同逻辑沿 z 轴扩展。画布的单位单元与 Minecraft 的单位方块相同，因此算法中的坐标——对应于游戏中的 (x, y, z) 坐标。

Minecraft 映射。如果玩家在 $(0, 64, 0)$ 出生且希望 $D = 16$ ($r = 8$) 的球位于地面，则世界坐标中的中心应放在 $(0, 72, 0)$ （地表上方 8 块），使南极恰好接触地面。对应的 WorldEdit 命令：在 $(0, 72, 0)$ 处执行 `//sphere stone 8`。

3. 基本方程：圆与椭圆

项目的所有理论始于以原点为中心、半轴为 r_x 和 r_y 的椭圆隐式方程：

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

当 $r_x = r_y = r$ 时，椭圆退化为半径 r 的圆周。内部的点满足 ≤ 1 ；外部的点满足 > 1 。该不等式决定体素是否属于形状，进而决定用户是否在该处放置方块。

将中心平移到 (c_x, c_y) 并在每个单元的中心进行评估：

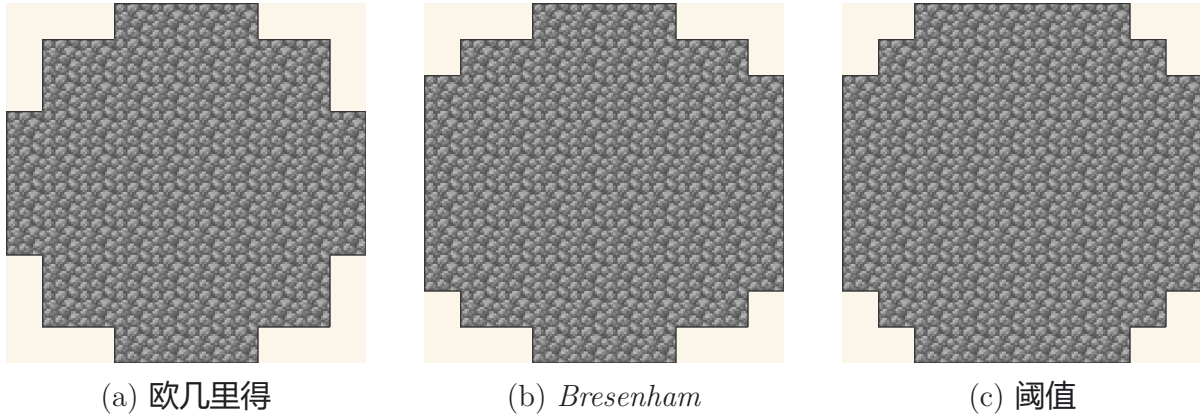


Figure 1: 在三种判定下直径 $D = 10$ 的同一圆周。每幅图显示对应版本的不等式 $d_x^2 + d_y^2 \leq 1$ 所标记的单元集合, 使用 Minecraft 的 *cobblestone* 材质渲染。注意行间过渡、角点单元和基本方位的差异——三种在不同判定下都数学正确的答案。

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{内部} \iff d_x^2 + d_y^2 \leq 1$$

Minecraft 映射。在生存模式中建塔时, 用户通常希望基础是宽而扁的圆, 向上逐渐变小。每层楼都是同一隐式椭圆方程的应用, 半轴相同但 z 不同。Block Round 一次计算 2D 层, 用户通过每层 `/clone` 命令复制该层。

4. 欧氏算法 (距离测试)

思路。对每一行 j , 解析地找出位于椭圆内部的列 i 。在给定 y 时求解 x :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

对于垂直位移为 $d_y = j + \frac{1}{2} - c_y$ 的行 j , 椭圆在该高度的水平半宽为:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

若 $d_y^2/r_y^2 \geq 1$, 则该行不与椭圆相交, 跳过。否则填充所有满足 $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ 的列 i 。整数边界来自:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

论证。项 $+\frac{1}{2}$ 来自单元 i 中心在 $i + \frac{1}{2}$ 的约定。函数 $\lceil \cdot \rceil$ 和 $\lfloor \cdot \rfloor$ 将连续区间转换为整数索引，确保仅包含**中心**位于椭圆内部的单元。该构造产生最接近连续曲线的离散近似，因而轮廓视觉最平滑。但对于小直径，结果可能不那么像素艺术——一个 $D = 5$ 的欧氏圆只有 13 个方块，而同直径的阈值圆有 21 个。

Minecraft 映射。欧氏准则最接近 WorldEdit 的 `//sphere` 所产生的结果。建造 $D = 16$ ($r = 8$) 的圆石球仅用欧氏外壳时，实心模式体积约 2145 块，细模式 (外壳) 约 804 块——即采集 34 堆 (2176) 与 13 堆 (832) 的差别。

5. Bresenham 算法 (整数中点)

Bresenham 算法于 1965 年发表用于绘制线段，后被推广到圆，再由 Pitteway 和 Kappel 推广到任意椭圆。其显著性质是无需浮点运算和开方：下一像素的确定仅使用**整数加法与比较**，通过一个决策函数评估两候选像素之间的中点位于解析曲线的哪一侧。对 Minecraft 建造而言，这是产生最易识别像素艺术外观的算法——自 2011 年起社区手工建造时使用的同样的阶梯形轮廓。

5.1 圆形情况

对于整数半径 R 的圆，从 $(x=0, y=R)$ 开始，初始决策参数为：

$$d_0 = 1 - R$$

每一步 (在 0 到 45° 的八分圆中)：

$$\begin{cases} \text{如果 } d < 0: & \text{下一个是 } (x+1, y), \quad d += 2x + 3 \\ \text{如果 } d \geq 0: & \text{下一个是 } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

论证。函数 d 在每次迭代中近似在两个候选像素的**中点**处计算的圆隐式方程的值：

$$F(x+1, y - \frac{1}{2}) = (x+1)^2 + (y - \frac{1}{2})^2 - R^2$$

d 的符号指示中点位于圆内 (仅水平推进) 或圆外 (同时递减 y)。圆的八个八分圆通过将二面体群 D_4 的对称性应用于计算的八分圆得到，对应代码中的八次 `span(...)` 调用。结果轨迹呈现离散逼近光滑曲线的典型阶梯形模式。

5.2 椭圆情况（两个区域）

对于半轴为 a, b 的椭圆，算法将第一象限分为**两个区域**，由切线为 45° 的点分隔：

区域 I: $2b^2x < 2a^2y$ ($|dy/dx| < 1$), 区域 II: 否则

初始决策参数为：

$$p_1 = b^2 - a^2b + \frac{a^2}{4},$$

$$p_2 = b^2\left(x + \frac{1}{2}\right)^2 + a^2(y - 1)^2 - a^2b^2.$$

每次迭代中， p 由预计算的增量更新（乘以 4 后全部为整数）。结果是每像素的最小算术：一次符号检测和两次加法。

Minecraft 映射。对生存玩家而言，Bresenham 轮廓常优于欧氏轮廓，因为离散步骤易于手动计数——用户知道下一个方块永远是直或对角。 $D = 11$ 的 Bresenham 圆有数十份社区指南中发布的标准 80 块序列。

6. 阈值算法（角点覆盖）

该算法询问：单元的任何角点是否在椭圆内？对每个单元 (i, j) ，代码寻找离形状中心最近的角点（中心在单元边上的投影）：

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{内部} \iff \left(\frac{n_x - c_x}{r_x}\right)^2 + \left(\frac{n_y - c_y}{r_y}\right)^2 \leq 0,94$$

值 0,94 是略低于 1,0 的经验阈值。其目的是消除在四个方位点（北、南、东、西）出现的 1 像素切向尖刺伪影，该处曲线与单元的一整条边相切。

在三种算法中，这种算法在相同直径 D 下产生面积最大的轮廓，因为包含条件最不严格。对 Minecraft 而言，这是最圆润外观的算法，但代价是更多方块—— $D = 16$ 在阈值下产生 213 个外壳块，Bresenham 下 200 个，欧氏下 192 个。

Minecraft 映射。当建造远观时（例如神庙顶上的石英穹顶），阈值是首选，因为略大的轮廓在环境遮蔽阴影下读起来更清晰。代价是每外壳约比欧氏多 5–10% 的方块。

7. 渲染模式：实心、细、粗

有了内部单元的矩阵 $f[j][i] = 1$ ，项目通过**离散拓扑**导出三种视觉风格：

7.1 实心

返回未改的 f 。所有内部单元都是方块。这就是 `//sphere stone 8` 所产生的结果。

7.2 细（轮廓）

如果一个单元被填充且至少有一个正交邻居（北、南、东、西）位于形状之外，则它属于细轮廓。逻辑记号：

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

几何上：这是形状的**离散边界**，是拓扑边界 $\partial F = F \cap \overline{F^c}$ 的离散类比。在 Minecraft 中等同于 `//hsphere`：仅外壳被物化。

7.3 粗

粗模式在边界像素上添加**对角桥**：同时具有水平边界邻居和垂直边界邻居的内部单元。这关闭了细模式留下的 1 像素对角线，当轮廓必须在场景中显得稳定时（无 45° 洞）很有用——对玻璃和冰圆顶尤其重要。

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Minecraft 映射。对于 $D = 20$ 的玻璃圆顶（中空），粗模式产生 608 块的密封外壳，细模式约 510 块——多出的约 100 块是对角桥补充了细模式留下的对角空体素，防止环境光从这些空气格漏入。

8. 离散面积计算

`area2D` 函数简单地**计算** f 中标记的单元数。这就是计数测度，它近似椭圆指示函数的 Riemann 积分：

$$\pi r_x r_y \begin{matrix} \text{连续面积} \end{matrix} \longleftrightarrow \sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} \begin{matrix} \text{离散面积} \end{matrix} f(i, j)$$

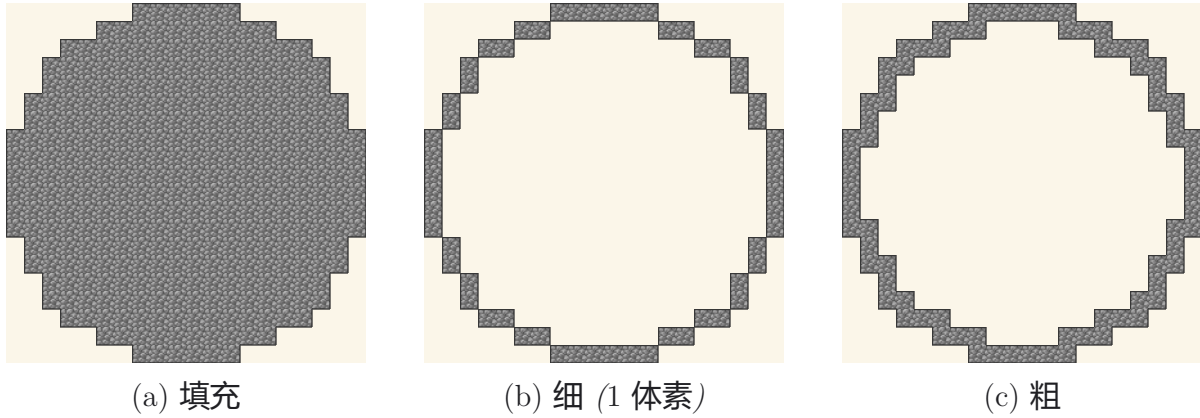


Figure 2: 同一 $D = 20$ 欧几里得圆周的三种渲染模式。细模式对应离散拓扑边界 (∂F)；粗模式增加对角块以封闭 45° 的孔洞, 这对密封构造 (水下玻璃/冰穹顶) 至关重要。

对大 D , 离散面积/连续面积 $\rightarrow 1$ 。对小 D , 算法间差异可见: 阈值倾向于高估; 欧氏接近理想面积; Bresenham 介于两者之间。

Minecraft 映射。 Block Round UI 中的方块计数芯片显示正是这个数: $D = 8$ 时 268, $D = 12$ 时 904, $D = 16$ 时 2145。该芯片以 N (堆 $\times 64$ + 余) 表达计数——因此 $2145 = 33 \times 64 + 33$, 即“33 堆加 33 个”。

9. 从 2D 到 3D: 椭球方程

3D 中, 基本形状是以 (c_x, c_y, c_z) 为中心、半轴 r_x, r_y, r_z 的**椭球**。其方程是椭圆的自然推广:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

当 $r_x = r_y = r_z$ 时, 回到**球**。在每个体素中心进行评估:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{内部} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Minecraft 映射。 $W = 20$ 、 $H = 10$ 、 $D = 20$ 的椭球产生扁平球, 适合地下圆顶或体育馆顶。体积约 2094 块 (33 堆)。等效 WorldEdit 命令: `//ellipsoid stone 10 5 10`。

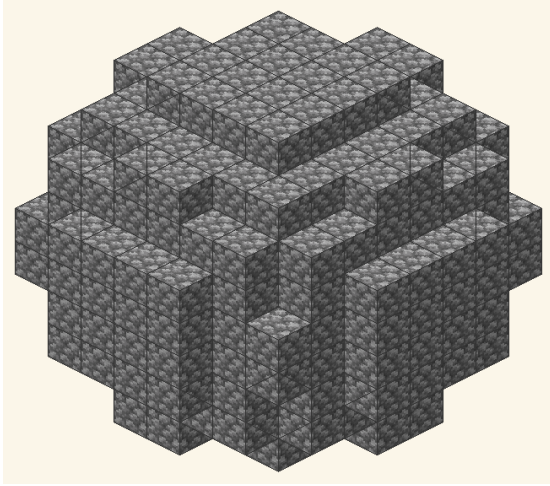
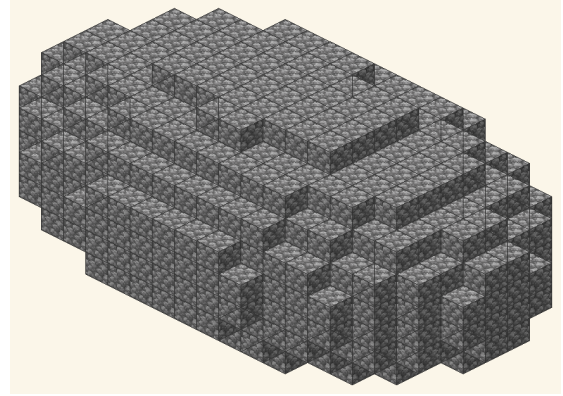
(a) 球体 $D = 10$, $r_x = r_y = r_z = 5$.(b) 椭球体 $W = 20$, $H = 10$, $D = 12$.

Figure 3: $\mathbb{R}^3$ 中的体素化。每个立方体是一个体素; 以其中心坐标 (i, j, k) 代入隐式方程 $a_x^2 + a_y^2 + a_z^2 \leq 1$ 进行检验。表面上可见的阶梯是离散化的数学必然结果——正是赋予 Minecraft 视觉标识的特性。

10. 体素化与体积计算

椭球的连续体积是积分学的经典结果, 通过对圆盘积分得到:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

离散版本遍历盒子 $D_x \times D_y \times D_z$ 的每个体素, 当 $a_x^2 + a_y^2 + a_z^2 \leq 1$ 时递增计数器。这就是 `voxelVolume` 算法。

外壳与内部。 对 *filled* 模式, 项目仅显示至少有一个 6-邻居在保留集合之外的体素 (即从外表面或切面可见的体素)。对 *thin*, 仅显示完整椭球的**表面** (1 体素厚度)。在生存中仅可见面需要其专用的有纹理方块——内部体素可以是廉价填充 (如用泥土代替钻石) 以节省资源。

参考表。 下表汇总 Minecraft 社区最常请求的标准球直径, 翻译为方块数 (V)、64 堆数 ($P = \lceil V/64 \rceil$) 和 $54 \times 64 = 3\,456$ 格双箱数 ($DC = \lceil V/3,456 \rceil$)。建造前用此表规划资源采集。

直径	方块 (V)	堆 ($\times 64$)	双箱	备注
D=8	268	5	1	小圆顶
D=12	904	15	1	中等球体
D=16	2145	34	1	大塔顶
D=20	4189	66	2	体育馆顶
D=24	7238	114	3	大圆顶
D=32	17156	269	5	超大型建造

在 $D \in \{8, 12, 16, 20, 24, 32\}$ 下用欧氏算法计算的实心模式体积。细模式 (外壳) 值约为 $V \approx 4\pi r^2$ 块 (1 体素厚), 即根据 D 在实心体积的 25% 到 40% 之间。

11. 几何切割: 平面与 45° 对角切割

Block Round 允许通过**三个平面**切割实体。每个切割是一个线性不等式, 丢弃体素:

$$\begin{aligned} \text{X 轴: } x &\geq cut &\Rightarrow \text{丢弃} \\ \text{Y 轴: } y &\geq cut &\Rightarrow \text{丢弃} \\ \text{对角: } x + y &\geq cut &\Rightarrow \text{丢弃} \end{aligned}$$

X 和 Y 平面垂直于坐标轴, 法向量分别为 $(1, 0, 0)$ 和 $(0, 1, 0)$ 。对角平面法向量为 $(1, 1, 0)/\sqrt{2}$, 在 XY 平面切 45° : 数学上是平面族 $x + y = k$ 。由于 $D_x \times D_y$ 盒中 $x + y$ 的最大值是 $D_x + D_y$, 对角切滑块范围为 $D_x + D_y$, 而 X 和 Y 分别为 D_x 和 D_y 。切割是**比例的**: 代码存储百分比 $cutPct$ 并重新计算极限:

$$cut = cutPct \cdot \max_{\text{轴}}$$

使得在一个轴上的 50% 在用户切换轴或调整大小时仍为 50%。

Minecraft 映射。球上 Y 轴 50% 切割产生标准的圆顶, 体积减半。对 $D = 16$: $V_{\text{full}} = 2145$ 块, $V \approx 1095$ 块 (约 18 堆)。50% 的对角切割产生半碗截面, 常用于喷泉内部和圆形剧场座位。

12. 粗 3D 模式: 通过切片的三维化

对 3D 中的 *thick* 模式, 项目沿三个轴对**所有切片**应用 2D 粗算法: XY 对每个 z , XZ 对每个 y , YZ 对每个 x 。然后取结果的并集。几何动机是不漏水性: 堵住所有对角而不加倍外壳厚度的最小体素集。形式上, 设 $S_z(z)$ 为高 z 处的 XY 切片,

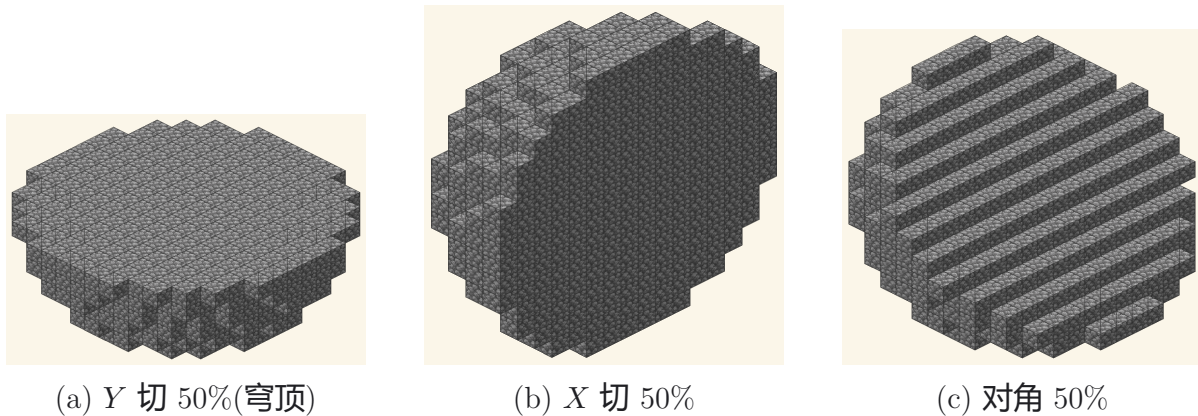


Figure 4: 对同一 $D = 16$ 球体的三种切法, 均保留 50% 体积。Y 切产生经典穹顶 $V = \frac{2}{3}\pi r^3$; 对角切暴露出该切割平面特有的斜边 $x + y = k$ 。

T 为 2D 粗算子:

$$thick3D = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

结果随后与切割保留的集合相交。背后理论是**数字拓扑**: 粗算子保证外壳的 26-连通性, 对 Minecraft 建造有用, 因为对角相邻体素不通过面相接——光、水和生物穿过这些对角的空体素 (即空气方块) 就像墙不在那里。

Minecraft 映射。对于生存模式中海底神殿上的水下玻璃圆顶, 粗模式是强制的: 对角桥填补了细模式留下的对角空体素 (空气方块), 从而阻止水从这些空气格渗入。代价: $D = 20$ 粗模式圆顶约需 380 块玻璃, 对比细模式约 320 块——多出 19% 材料换取完全密封。

13. 3D 相机: 球坐标

3D 相机以距离 r 围绕物体运行, 有两个角度—— θ (方位角, 水平平面) 和 φ (极角, 从垂直轴)。这是球坐标的**物理约定**, 转换为笛卡尔坐标 (three.js 所期望的) 为:

$$\begin{aligned} x &= r \sin(\varphi) \cos(\theta), \\ y &= r \cos(\varphi), \\ z &= r \sin(\varphi) \sin(\theta). \end{aligned}$$

初始位置: $\theta = \pi/4$ (45° , 等距视角) 和 $\varphi = \pi/3$ (从北极 60° , 略高于赤道)。鼠标拖动直接增量 θ 和 φ ; 滚轮/捏合增量 r 。这种参数化的简单性允许自由旋转, 无需四元数或显式矩阵。

14. 自动缩放：包围球与 FOV

当用户改变图形大小或重置相机时，项目重新计算**理想距离**，使物体在任何角度都适合视口。思路是将物体包含在半径 R 的球中（AABB 对角线的一半，轴对齐包围盒）：

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

球具有**旋转不变投影**，因此如果在一个方向适合视口，则在所有方向都适合。给定相机的垂直 FOV ($fov_V = 35^\circ$ 弧度)，框住半径 R 的球所需距离为：

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

水平 FOV 由画布纵横比给出：

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

最终距离是 $dist_V$ 和 $dist_H$ 的最大值乘以 1.20（20% 视觉边距）。当图形被切割时，代码将 D_x 或 D_y 缩到剩余大小。当橡树或苦力怕彩蛋激活时，包围盒向上扩展以包括实体高度。

15. 阴影与纹理乘法

三种 3D 风格（*Classic*、*Smooth*、*Blocks*）通过应用于每个体素三个可见面（顶部、亮侧、暗侧）的**乘法因子**区分。函数 `shadeHex(hex, factor)` 将十六进制解析为 (R, G, B) 并对每个通道相乘，钳在 $[0, 255]$ ——结果成为应用在方块原始纹理之上的色调：

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

这是理想 Lambert 光照函数的线性近似，

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \quad ,$$

三个因子对应每个面法线与固定光照方向夹角的余弦。在 *Smooth* 中因子相近（面相似）；在 *Classic* 中因子差异更大（强对比，原版 Minecraft 的外观）。*Blocks* 额外引入几何内缩——每个面向内的常数平移，以揭示体素间的边界，即使相邻体素带相同纹理。对发光方块（萤石、海晶灯、菌光体、岩浆、哭泣的黑曜石）Lambert 项被常数 $I = \quad \cdot$ 替换。

16. 过渡：从连续数学到游戏内建造

以上十五节描述了 Block Round 与姊妹项目 Pixel Round 共享的连续到离散管线：从椭圆隐式方程一直到框定结果的相机。剩余六节描述 Block Round 所**特有**的内

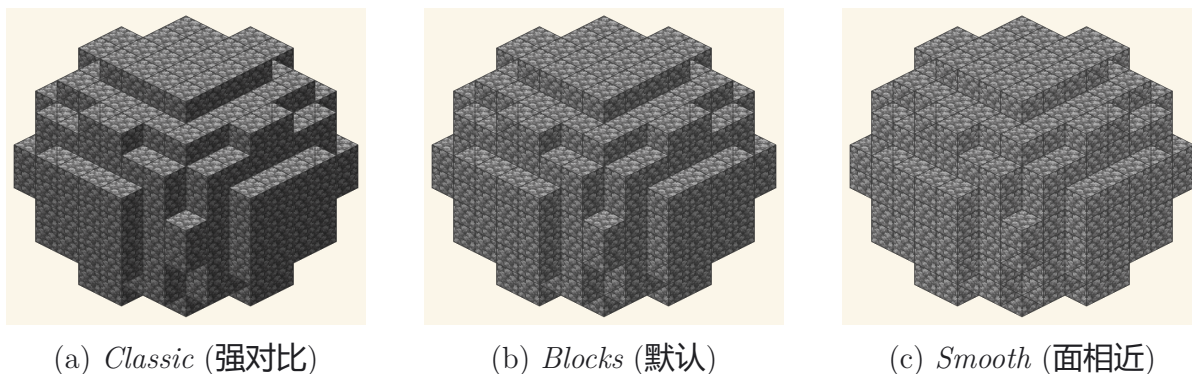


Figure 5: 对同样三个可见面应用三种光照倍数。*Classic* 给出标志性的 Minecraft 原版外观;*Smooth* 降低对比度以呈现平滑表面;*Blocks* 为中间默认值, 其几何内嵌即使在材质相同时也能显露体素边界。

容——由于渲染单元不是抽象像素而是 *Minecraft* 方块所引入的额外数学层, 每个方块有六个有纹理的面、在物品栏中有重量、有采集和放置的时间成本。

这些不是装饰性考虑。决定用方块纹理渲染改变了用户面临的操作问题: 不是导出 *PNG*, 而是将轮廓转换为放置计划。接下来几节的数学内容正是关于这种转换: 物品栏算术、面计数优化、逐层分解, 以及利用对称性将 N 次方块放置的成本减少到 $N/8 + 7$ 次命令调用。

17. Minecraft 物品栏算术

一个 Minecraft 玩家在 36 格物品栏中携带物品，组织为 9×4 网格。每格最多容纳 64 个同类物品（一堆或一包）。玩家也可访问储存：单箱 27 格，双箱 54 格。将目标方块数转换为采集计划的算术因此是向上取整除法问题。

17.1 单格、单堆、满包

堆是物品栏会计的基本单位。每堆至多 64 个物品。从方块数 N 到包数（或堆数）的转换为

17.2 储存：箱子、双箱、潜影盒

要计算建造需要多少双箱，将方块数除以 $54 \text{ 格} \times 64 \text{ 个/格} = 3456 \text{ 个/双箱}$ ：

$$\text{堆： } N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil \quad \text{双箱： } N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$$

单箱容 $27 \times 64 = 1728$ 个。潜影盒（便携容器）容 $27 \times 64 = 1728$ 个，但本身可放进箱格，因此装满 54 个潜影盒的双箱可容 $54 \times 1728 = 93312$ 个。对超大型建造，相关容量单位是装满潜影盒的双箱。

17.3 参考表：标准球

下表将标准实心球体积转换为采集计划。最后一列建议每个直径的建造场景：

直径	V (方块)	堆数	储存	典型建造
D=8	268	5	1 双箱	小屋屋顶，水井圆顶
D=12	904	15	1 双箱	瞭望塔顶
D=16	2145	34	1 双箱	村庄钟，图书馆圆顶
D=20	4189	66	2 双箱	天文台圆顶
D=24	7238	114	3 双箱	神庙圆顶
D=32	17156	269	5 双箱	大教堂圆顶，世界 boss 竞技场

Block Round 的信息芯片显示的堆-余数表示 $N = q \cdot 64 + r$ 反映了生存物品栏显示部分堆的方式：玩家看到 q 个满格加一个显示 r 的格。对 $D = 16$ ，芯片打印“2 145 (33 $\times$ 64 + 33)”，告诉玩家完全填充 33 格再加一格 33 个——共 34 格。

18. 高亮覆盖层与裸露面计数

Block Round 默认在每个可见体素面的边缘渲染黑色高亮覆盖层。视觉上类似游戏内调试覆盖层的“F3+G”区块边界风格，缩到每体素级别。数学上覆盖层实现**裸露面计数**的可视化：生存玩家用以在建造时逐块验证进度的数字。

18.1 什么算裸露面

体素 v 与邻居 n 共享的面是裸露的当且仅当 $n \notin V_{\text{solid}}$ (邻居为空)。每个实心体素贡献 0 (完全埋藏) 到 6 (孤立) 个面。总裸露面数 F_{exp} 是图形表面积的离散类比。

18.2 边界检测公式

体素位于边界，当且仅当其六个面邻居中至少一个不在实心内：

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

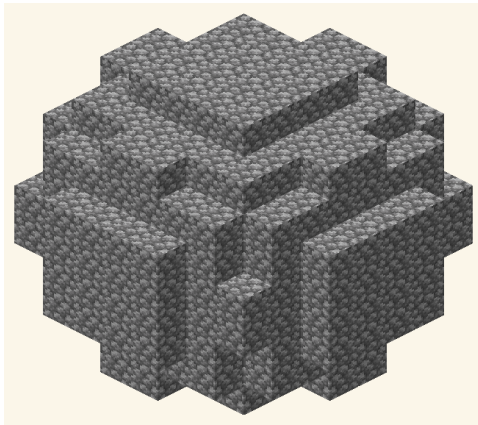
这是 2D 细算法的同一边界算子推广到 3D，覆盖层简单地绘制每个边界体素的立方体边线。对玻璃和冰默认为关；对圆石、石头、钻石及其他不透明方块默认为开，因为覆盖层将内部体素与外壳消除歧义。

18.3 裸露面计数作为生存启发法

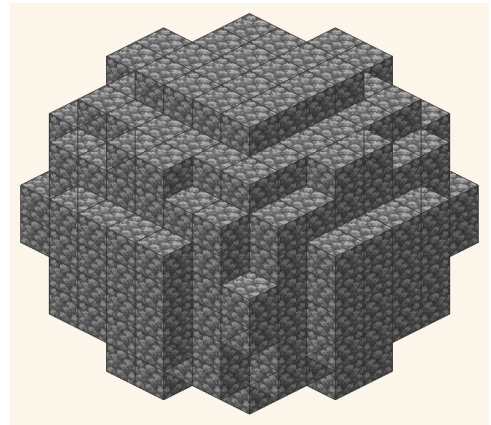
对生存玩家，相关计数不是总体积 V 而是裸露面数 F_{exp} ，因为这是从外部可见的。所有实固体素上 F_{exp} 之和等于六倍边界体素数减去两倍边界体素间的面相邻数：

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

对 $D = 16$ 球（实心模式）， $V = 2145$ ， $V \approx 432$ ， $F_{\text{exp}} \approx 1184$ 个可见面——建造只需约 432 个可见方块；其余 1713 块是内部，可用最便宜的方块（泥土、圆石）填充。覆盖层让玩家立即发现外壳中遗漏的方块。



(a) 叠加 OFF



(b) 叠加 ON(默认)

Figure 6: 同一体素集合在外露棱上有/无黑色轮廓的对比。开启 *highlight overlay* 时，外壳上缺失的方块会产生肉眼一瞥即可察觉的图案不连续——生存建造中的快速验证工具。

19. 逐层建造（水平分解）

在 Minecraft 中建造按水平层进行：玩家站在一层上，放置上一层的方块，然后上升一格。3D 椭球问题因此最好理解为 2D 椭圆的堆叠，每层一个：

$$\text{层}(k) = \left\{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \right\}$$

对每个整数层 k 从 $-r_z$ 到 $+r_z$ ，横截面本身是半轴缩小的椭圆 $r_x(k)$ 和 $r_y(k)$ ：

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

这意味着 3D 问题被简化为一次一层地计算前几节的 2D 算法，并堆叠结果。认知上是巨大简化：玩家不必心中跟踪三个坐标，而是每层跟踪两个坐标加层索引。

Minecraft 映射。对 $D = 16$ ($r_z = 8$) 的球, 赤道层 ($k = 0$) 是完整的 Bresenham 圆 $D = 16$ (约 200 块)。上一层 ($k = 1$) 是 $D = 15,97$ 的圆。 $k = 4$ 时层是 $D = 13,86$ 的圆 (约 148 块), $k = 7$ 时缩为 $D = 7,48$ 的圆 (约 43 块)。两极 ($k = \pm 8$) 是一个方块的盖子。

Block Round 的信息芯片在 3D 模式下当用户激活中心引导按钮 (键 c) 时显示这种逐层计数。水平切片在赤道和每个 $r_z/2$ 细分处绘制为半透明黄色十字。

20. 通过八面体对称性的优化 (O_h)

以原点为中心的球在 **八面体群** O_h (阶 48) 的作用下不变。球的体素化保留一个阶 48 的子群, 由三个坐标反射和绕每轴的三个 $\pi/2$ 旋转生成。对实际建造目的, 相关子群是阶 8 的八分体反射群 $\mathbb{Z}_2^3$, 由 $x \mapsto -x$ 、 $y \mapsto -y$ 、 $z \mapsto -z$ 生成。一个八分体的体素集决定整个球:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \Rightarrow \text{减少} = \frac{N - N/8}{N} = 87,5\%$$

这是 Minecraft 玩家可用的最大数学优化。不是手动放置 N 块, 而是在正八分体放置 $N/8$ 块, 然后发出七条 `/clone` (原版) 或 `//copy + //paste` (WorldEdit) 命令, 带适当反射。

20.1 具体命令序列

对世界坐标 $(0, 72, 0)$ 为中心的 $D = 24$ ($r = 12$, $V = 7\,238$ 块) 球, 先建造正八分体 $(+x, +y, +z$ 区域, 约 905 块), 然后发出:

```
/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked    (-x 反射)
/clone 0 72 0 12 84 12 0 60 0 mode:masked      (-y 反射)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked    (-z 反射)
/clone -12 72 0 -1 84 12 mode:masked            (-x, -y 八分体)
/clone 0 60 -12 12 71 -1 mode:masked            (-y, -z 八分体)
/clone -12 72 -12 -1 84 -1 mode:masked         (-x, -z 八分体)
/clone -12 60 -12 -1 71 -1 mode:masked         (-x, -y, -z 八分体)
```

每个 `/clone` 约花费 50 ms 服务器时间, 因此七次反射在不到半秒内完成。手动放置八分体的 905 块需约 15 min (生存) 或 45 s (创造, 含刷子)。完整放置 7 238 块 (无对称) 需约 2 h (生存) 或 6 min (创造)。

20.2 时间成本对比

设 T_{block} 为每块放置时间（典型生存 1–2 s，创造 0,1–0,5 s）， T_{clone} 为每条克隆命令时间（约 50 ms）。有无对称的总建造时间为：

$$T_{\text{octant}} + 7T_{\text{clone}} \ll T_{\text{full}}$$

对生存中 $N = 7238$ ($T_{\text{block}} = 2\text{ s}$, $T_{\text{clone}} = 0,05\text{ s}$)：完整 = 14476 s $\approx$ 4 h 1 min；八分体 + 克隆 = 1810 s + 0,35 s $\approx$ 30 min。8 倍加速恰好对应反射子群的阶。完整阶 48 的 O_h 原则上可进一步减少（48 倍），但利用对角旋转需 45° 旋转的建造区域，在原版 Minecraft 中很少值得设置成本。

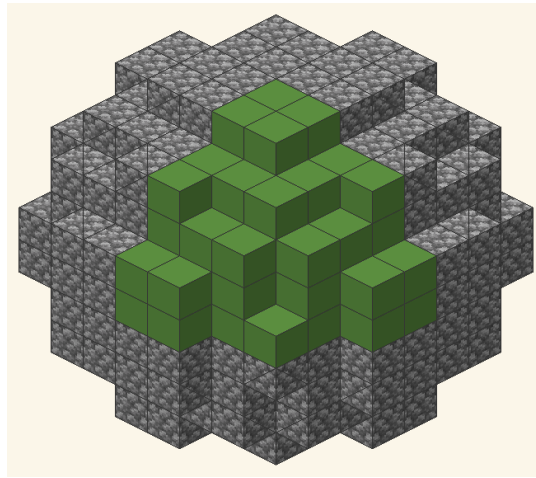


Figure 7: $D = 10$ 球体, 正八分卦象 $(+x, +y, +z)$ 以绿色高亮。在子群 $\mathbb{Z}_2^3 \leq O_h$ (阶 8, 由三个坐标反射生成) 的作用下, 该八分卦象决定整个球——其余七个区域通过对手工构建的八分卦象施加一系列带 *mode:masked* 的 `/clone` 调用得到。

21. 方块纹理与视觉构图

Minecraft 方块不是平面着色单元，而是六个面有纹理的立方体。Block Round 用方块的六个标准面纹理渲染每个体素，从原版资源包采样。数学上纹理操作是 UV 映射：单位立方体的每个面由 $[0, 1]^2$ 坐标参数化，相应纹理在这些 UV 上采样以产生渲染像素。

21.1 六个面与 UV 映射

对每个体素面，渲染器查询方块模型：

$$\text{面} \in \{\text{顶, 侧, 底}\} \quad UV : [0, 1]^2 \rightarrow \text{pixels}$$

大多数方块（石头、圆石、钻石块、橡木板）六面同纹理。多面方块（草方块、南瓜、干草捆、石英柱、所有原木、工作台、熔炉、书架、TNT）使用不同纹理：草方块顶绿、侧草混土、底纯土；橡木原木顶底带年轮、四侧带树皮。关键的是，放置哪些体素的数学不依赖所用纹理——钻石球和圆石球同直径的轮廓完全相同。纹理决策是几何算法完成后应用的纯审美层。

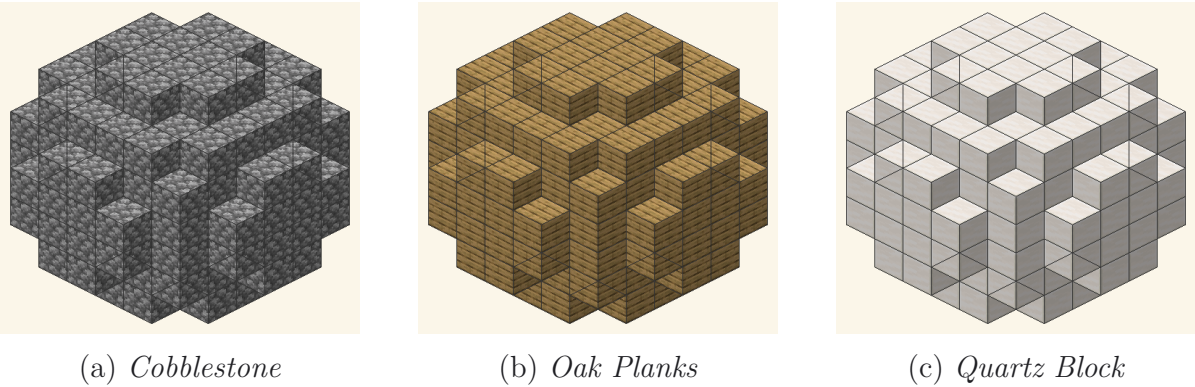


Figure 8: 同一 $D = 8$ 球体使用三种不同方块渲染。体素化轮廓 (数学) 在三者中完全相同: 算法选定体素; 材质是事后施加的纯粹美学层, 既不改变体积也不改变拓扑。

21.2 方块家族与色调

对生存玩家从数十种可用方块中选择，相关问题不是纹理索引而是色调家族：浅/深、暖/冷、均匀/有纹样。下表分类 Block Round 建造中最常用的方块：

方块	家族	色调	典型用途
cobblestone	石	灰	乡村墙体，地牢
stone	石	灰	洁净墙体，台座
oak_planks	木	米	地板，内饰
quartz_block	石英	白	神庙，现代圆顶
sandstone	砂	米	沙漠，金字塔
deepslate	石	暗	地下，点缀

21.3 通过方块混合的渐变

尽管单一方块类型给出一个色调，渐变可通过每层混合两到三种方块得到。一种经典 Minecraft 技术是圆顶上的石-石英渐变：下层是平滑石头，赤道层是平滑石头与闪长岩的噪声混合，上层是石英。每层仍由相同的 Block Round 算法计算；变化的是每体素纹理查找。Block Round 的 *Random* 方块选项默认实现这种程序化混合（草顶，下面泥土，中间稀疏矿石，靠近基岩有深板岩）。数学与第 19 节的逐层分解相同；只是每体素的“方块标签”变化。

22. 结论

本文系统地描述了 Block Round 中所用的数学基础。在约一千行 JavaScript 代码加上配套的纹理与音频管线中，项目整合了若干理论领域的贡献：

- **解析几何**：椭圆和椭球的隐式方程作为主要的归属准则。
- **经典光栅化算法**：中点形式的 Bresenham，含针对椭圆的 Pitteway/Kappel 扩展。
- **数字拓扑**：离散边界算子，4-、6-、26-连通概念，切片组合与裸露面计数。
- **积分学**：椭球体积作为三重积分及其离散对应（体素上的计数测度）。
- **线性代数**：由线性不等式定义的切割平面与三维透视投影。
- **三角学**：球坐标中的相机参数化与依赖 FOV 的自动距离调整。
- **组合物品栏算术**：64 堆和 3456 双箱的向上取整除法会计，含信息芯片显示的堆-余数表示。
- **群对称性**：利用八面体群 O_h 及其阶 8 反射子群 $\mathbb{Z}_2^3$ 通过 `/clone` 将建造成本减少 8 倍。

研究允许阐述的核心观察是：在离散网格上，连续曲线**不存在唯一正确的表示**。本文给出的每种算法对应于单元包含准则的一个独特数学定义，每个定义产生具有自身形式属性的轮廓——欧氏的平滑、Bresenham 的阶梯模式、角点覆盖准则的更大覆盖。算法选择因而是技术决策，必须考虑预期的视觉表示、所需的度量特征，以及——特别是对 Block Round ——在游戏中物理建造形状的物品栏和时间成本。

Block Round 在 Minecraft 工具生态系统中占据小但数学上丰富的一块：位于纯几何工具（WorldEdit、Litematica）和纯视觉工具（资源包、着色器）之间，提供从连续方程到离散放置计划的明确、可推导、可重现的桥梁。姊妹项目 Pixel Round 提供相同算法但没有 Minecraft 层，适合传统像素艺术和体素艺术应用。两项目共同例证同一连续到离散数学管线如何支持截然不同的艺术和构造 workflow。

参考文献

技术参考 (光栅化与数字拓扑)

- **Bresenham, J. E.** *Algorithm for computer control of a digital plotter*. IBM Systems Journal, vol. 4, n. 1, p. 25–30, 1965. 可获取于 [doi:10.1147/sj.41.0025](https://doi.org/10.1147/sj.41.0025). — 本文档第 §5 节所讨论的整数中点算法的原始论文。
- **Pitteway, M. L. V.** *Algorithm for drawing ellipses or hyperbolae with a digital plotter*. The Computer Journal, vol. 10, n. 3, p. 282–289, 1967. 可获取于 [doi:10.1093/comjnl/10.3.282](https://doi.org/10.1093/comjnl/10.3.282). — 将 Bresenham 算法推广至任意圆锥曲线, 是第 §5 节中椭圆扩展的基础。
- **Kappel, A.** *An ellipse-drawing algorithm for raster displays*. 收录于 *Fundamental Algorithms for Computer Graphics* (R. A. Earnshaw, ed.), NATO ASI Series F-17, Springer, 1985, p. 257–280. — 椭圆实现中所采用的两区域公式。
- **Klette, R.; Rosenfeld, A.** *Digital Geometry: Geometric Methods for Digital Picture Analysis*. Morgan Kaufmann, 2004. 656 页. — 数字拓扑、边界算子 (∂F)、4-/6-/26-连通性的标准参考。用于 §7、§12 和 §18。
- **Foley, J. D.; van Dam, A.; Feiner, S. K.; Hughes, J. F.** *Computer Graphics: Principles and Practice*. 第 3 版, Addison-Wesley, 2014. — 光栅化、光照模型、透视投影的经典论述。与 §13–15 相关。

所引用的 Minecraft 规范与工具

- **Sponge Project.** *Sponge Schematic Specification, version 2*. github.com/SpongePowered/Schematic-Specification. — Block Round 导出的 `.schem` 文件格式 (gzip + NBT)。可由 WorldEdit、Litematica 和 MCEdit 加载。
- **EngineHub.** *WorldEdit Documentation*. worldedit.enginehub.org. — 全文中引用的 `//sphere`、`//hsphere`、`//ellipsoid`、`//cyl`、`//schem load` 命令的官方参考。
- **Mojang AB.** *Minecraft Wiki — Inventory*. minecraft.wiki/w/Inventory. — §17 中所用关于槽位、堆叠、单箱 (27 槽) 和双箱 (54 槽) 的参考资料。
- **Mojang AB.** *Minecraft Wiki — Commands/fill, /clone*. minecraft.wiki/w/Commands/fill, minecraft.wiki/w/Commands/clone. — §17 与 §20 中引用的原版命令 `/fill` 与 `/clone`。
- **Masady (Litematica mod).** github.com/maruohon/litematica. — 导入示意图并以半透明幽灵显示待施工建筑的 Minecraft Java 版模组。

几何、线性代数与群论

- 王萼芳, 石生明 (编) 高等代数. 高等教育出版社, 北京, 2003. — 椭圆的隐式方程、 $\mathbb{R}^3$ 中的平面代数、球坐标。§3、§9 与 §13 内容的基础。
- Armstrong, M. A. *Groups and Symmetry*. Undergraduate Texts in Mathematics, Springer, 1988. — 八面体群 O_h 及其子群的入门论述, 用于 §20。
- Coxeter, H. S. M. *Regular Polytopes*. 第 3 版, Dover, 1973. — 正多面体对称群的经典参考, 包含 §20 中提及的 48 阶 O_h 的详细处理。
- Hilbert, D.; Cohn-Vossen, S. *Geometry and the Imagination*. AMS Chelsea, 1990 [orig. 1932]. — 二次曲面与几何可视化的经典论述, §9 的直观基础。

算法、数据结构、复杂度

- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 第 4 版, MIT Press, 2022. — §4–6 三种算法的复杂度分析, §17 的向上取整除法。
- Wirth, N. *Algorithms + Data Structures = Programs*. Prentice-Hall, 1976. — Bresenham 直线算法及其向圆周推广的经典论述。

补充在线资源

- Block Round — 应用程序: vinisouza128.github.io/block-round/.
- Block Round — 代码仓库: github.com/ViniSouza128/block-round.
- 姊妹项目 Pixel Round(无 Minecraft 材质版本): github.com/ViniSouza128/pixel-round.
- pt-BR 教案 (高中 3 年级): docs_aula/Plano_de_Aula_pt-BR.pdf.

附录 A：建造参考表与示例

本附录汇集了 Block Round 用户最常用的数值与命令参考。下文数据整合了文档中各节的表格和示例，组织为可作为建造规划、命令语法和端到端示例的单页查阅。

A.1 球体综合参考（实心、空心、堆数）

下表在第 10 节和第 17 节标准球表的基础上，将空心壳变体（细模式）并列于实心体积。空心壳计数假设单体素厚度，并在欧氏算法下计算；粗模式增加约 10–15% 的块以按第 12 节封堵对角缝。

D	实心 (V)	空心 (壳)	堆实心	堆空心
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

对生存玩家而言，空心列通常更相关：只有可见外壳需要专用纹理方块，内部（如果需要）可填充最便宜的材料。堆-实心列给出完全填充球的总储备量；堆-空心是空心圆顶或外壳的实际最小值。

A.2 WorldEdit / 原版命令参考

下表汇总在游戏内构造 Block Round 输出最相关的 WorldEdit 和原版 Minecraft 命令。WorldEdit 命令假设玩家手持工具（默认木斧）并选定区域；原版命令无需模组但需要坐标参数。

命令	模组 / 来源	结果
//sphere stone 8	WorldEdit	实心球 $r=8$ (欧氏)
//hsphere stone 8	WorldEdit	空心球 $r=8$ (仅壳)
//cyl stone 8 16	WorldEdit	圆柱 $r=8, h=16$
//ellipsoid st. 10 5 10	WorldEdit	椭球 $10 \times 5 \times 10$
/fill ... stone	原版	用一种方块填充立方体
/clone ... mode:masked	原版	将区域复制到新位置

对 Litematica 用户工作流不同：而不是运行命令，将 schematic 文件 (Block Round 导出 .schem, Sponge 格式 v2) 加载到 Litematica 模组中，该模组显示玩家逐块放置的图形的半透明幽灵。这是最接近 WorldEdit 创造体验的生存模式等价物。

A.3 示例：神庙上 $D=20$ 的石英圆顶

作为完整示例，假设玩家想在已有石头神庙顶上建造 $D = 20$ 的石英圆顶。神庙顶部为 11×11 块，中心在世界坐标 $(0, 80, 0)$ 。圆顶是半径 $r = 10$ 的球在 $Y = 0$ 平面切割得到的上半部分。建造计划如下：

- 用 Block Round 规划。** 打开应用，设 3D 模式、球体、大小 $D = 20$ 、切割轴 Y 为 50%。从选择器选石英块。信息芯片报告 $V \approx 2\,126$ 块 (约 34 堆，1 个双箱第二个剩 1330 件)。
- 收集资源。** $34 \text{ 堆石英块} = \lceil 2\,126/4 \rceil = 532 \text{ 石英块} \times 4 \text{ 下界石英每块} = 2\,128 \text{ 下界石英}$ 。冶炼或交易。
- 决定算法。** 从下方看的圆顶，欧氏算法给出最平滑轮廓；远山上圆顶则阈值算法读起来更整洁。Block Round 默认欧氏。
- 圆顶定位。** 圆顶平面必须与神庙顶部一致。包围盒为 $20 \times 10 \times 20$ 块；中心置于 $(0, 80, 0)$ ，平面（切割面）在 $y = 80$ ，圆顶顶点在 $y = 89$ 。
- 对称建造。** 建造 $+x, +z$ 象限 (约 530 块)。执行 `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` 沿 x 镜像；然后 `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` 沿 z 镜像。总用时：生存约 10 分钟。
- 润色。** 在顶点和赤道四个方位点添加海晶灯发光照明 (Block Round 用其发光贴图在 MC 光照等级 15 渲染)。可选切换 Block Round 边缘高亮 (键 g) 以在放置最后几块前验证圆顶外壳无缺。

每一步的数学正是前 22 节所描述的：椭球隐式方程、欧氏体素化、 Y 轴切割、通过 `/clone` 的八面体对称减少、把体积翻译为堆和箱的物品栏算术。附录因而不是新理论——它提醒文档每个理论概念都对应于建造规划工作流中一个单一、具体的决策。

A.4 键盘快捷键参考

Block Round 应用提供一组与画布角按钮对应的键盘快捷键。列在下方供快速查阅；快捷键全局生效（无需任何输入字段聚焦），在 2D 和 3D 模式下均可工作，除非另有说明。

键	切换	备注
G	边缘网格覆盖层	每个体素面上的黑色轮廓
C	中心引导线	坐标轴上半透明黄色十字
D	下载 PNG	当前画布为 PNG 图像
I	信息芯片	方块数和包细分
M	2D/3D 切换	平面与体素模式切换
S	声音开/关	环境放置方块声音
T	白天/夜晚主题	调暗背景，保持图块可读
Ctrl+Z / Y	撤销/重做	遍历图形修改历史

快捷键 **G**（网格）和 **C**（中心引导）在建造验证时尤其有用：**G** 显示外壳的每个体素是否就位（缺失方块会破坏覆盖层图案），**C** 确认切割正确居中在图形轴上。

对于 **Ctrl+Z** 撤销：Block Round 遍历每个改变图形的编辑（滑块、渲染模式、算法、形状、轴、方块、模式切换），但故意排除仅视觉切换（相机、边缘覆盖层、中心十字、主题、声音）。这种分离让撤销栈填充用户可能想要逆转的编辑，而非偶发视觉调整。

A.5 关键术语词汇表

文档中反复出现的技术术语的快速参考。每条简要回顾概念并指向其详细发展的章节。

- **体素 (Voxel)** — 3D 中的单位立方体，2D 像素的类比。每个 Minecraft 方块都是一个体素。参见第 2 节。
- **包 (Pack/堆)** — 在 Minecraft 中，占据一个物品栏格的 64 个相同物品的一堆。参见第 17 节。
- **双箱** — 两个相邻箱子合并为 54 格容器。最多容纳 3 456 个物品。参见第 17 节。
- **潜影盒 (Shulker box)** — 便携式 27 格容器，其本身可存储在箱子格内。参见第 17 节。
- **外壳** — 实心图形的外部单体素厚层。生存玩家的相关表面。参见第 7 节和第 18 节。

- **八分体** — 通过三个坐标半空间相交得到的 3D 空间的八个区域之一。用于对称优化。参见第 20 节。
- **UV 映射** — 3D 面的 2D 参数化，用于将纹理图像映射到体素的每个面上。参见第 21 节。
- **Bresenham** — 1965 年的增量光栅化算法，仅使用整数算术。参见第 5 节。
- **阈值** — 将单元包含的光栅化准则，前提是其任何一角在椭圆内。参见第 6 节。
- **欧氏** — 将单元包含的光栅化准则，前提是其中心在椭圆内。参见第 4 节。
- **彩蛋** — 由特定输入触发的隐藏功能。在 Block Round 中：橡树和苦力怕，二者均在滑块值 15。
- **Schematic** — 可由 WorldEdit、Litematica 或 MCEdit 加载的序列化结构文件 (Sponge Schematic v2, `.schem`)。

此词汇表故意保持比完整的数学词典短。需要更深入了解底层数学（数字拓扑、积分学、群论等）的读者，请参阅本文档的相关章节。

A.6 模型的展望与局限

本文件中提出的 Block Round 模型涵盖椭圆、椭球及其切割的静态体素化，以及 Minecraft 层引入的实用考虑。它不解决动态问题——移动图形、动画体素过渡、与流体模拟的交互——因为这些需要离散光栅化范围之外的连续时间框架。一个自然的扩展是 *schematic* 级动画：通过插值连接的静态体素化序列，Block Round 已经可以将其导出为 Litematica 脚本可读的 `.schem` 流。

另一个开放方向是保体积的精确体素化：当前的离散计数 V_{disc} 仅渐近收敛到连续体积 V_{cont} 。对于精确计数必须与连续公式匹配的应用（如渲染竞赛，对方块预算有严格要求的数学艺术），需要额外的精细化层——例如，略微调整半径使离散计数命中目标值。Block Round 目前不实现这一精细化，但底层数学很简单：数值求解 $V_{\text{disc}}(r) = V_{\text{target}}$ 对 r 。

最后，高阶对称利用：本文件涵盖 O_h 阶 8 的反射子群，但完整的阶 48 群包括旋转，原则上允许 $\times 48$ 加速。实际障碍是游戏内命令在轴对齐区域工作，利用旋转对称需要 45° 旋转的建造区域。内部旋转选择框的服务器模组（如 FAWE）原则上可以实现，但主流工具尚未公开。在原版服务器上演示 $\times 48$ 加速的参考实现，将是本工作的自然延续。

A.7 附加参考资料与外部资源

本文中阐述的数学概念各自有成熟的文献。对于寻求原始资料或更深算法背景的读者，推荐以下入口：

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 在本文档中扩展的整数算术光栅化原始论文。
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 将 Bresenham 推广到任意圆锥曲线。
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 第 5 节使用的两区域椭圆公式。
- R. Klette 与 A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 数字拓扑和第 7、18 节边界算子的标准参考。
- WorldEdit 文档, enginehub.org/worldedit/ 本文档中使用的 `//sphere`、`//hsphere`、`//ellipsoid`、`//copy` 系列命令的官方参考。
- Sponge Schematic 规范 v2, github.com/SpongePowered/Schematic-Specification Block Round 导出为 `.schem` 的文件格式。

第 22 节列出的八个理论领域各自有教科书级处理，远超单一技术文档所能总结的范围。Block Round 的贡献不在于理论本身，而在于具体综合：将经典光栅化应用于带纹理体素的 Minecraft 设置，并明确处理通常在纯几何工具中省略的物品栏和对称成本考虑。

关于可重现性的最后一句话：本文档的整个流水线 构建脚本、翻译、LaTeX 模板、按区域的表格 都包含在项目仓库的 `docs_math/` 目录中。在具有 XeLaTeX/MiKTeX 的系统上重新运行 `python build.py`，可以在所支持的九种区域中的任何一种中完全相同地产生此 PDF。姊妹项目 Pixel Round 遵循相同结构，允许直接对比纹理化和非纹理化变体。

文档结束

Block Round 由 Vinícius Rodrigues de Souza 维护。姊妹项目 Pixel Round 可在 github.com/ViniSouza128/pixel-round 获取；当前 Block Round 仓库位于 github.com/ViniSouza128/block-round。

欢迎通过项目仓库的 issue tracker 提交评论、修正和翻译改进。对八种非英语区域设置的母语审校特别受重视，因为本地化经过实质性审校，但技术 - 数学术语会因母语者修正而受益。